DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Design a **local swarm** for pure intelligence boost and capability enhancement in the tri/quad-node Sovereign Machina OS.
- The swarm must be spawned from **Node B (Director)** or **Node C (Oracle)** using the strongest local models (Gemma-4 Q8 on B, Gemma-4 E4B Q4 on C).
- Focus on intelligence amplification (reasoning depth, knowledge synthesis, self-improvement, pattern extraction, long-term memory evolution) rather than mission-building alone.
- Maintain: Node A = Mooncake KV buffer (unchanged), zero-trust, physical sovereignty, declarative Nix identity, local-first design, R3D_V01D + VT323 aesthetic.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (2026 swarm patterns + Sovereign Machina alignment)
MiroShark demonstrates that graph-centric, archetype-based swarms with background job queuing and live injection can scale intelligence through simulated interaction. However, MiroShark is Python-based and simulation-focused. For Sovereign Machina, we need a **native Rust/Tokio swarm** that runs entirely locally, leverages our existing Hermes supervisor and GBrain rewrite, and prioritizes pure intelligence boost (deep reasoning, relational synthesis, self-evolution).

RECOMMENDATION
Spawn the swarm from Node C (Oracle) as the primary entry point, with optional delegation to Node B (Director) for high-quality synthesis.

This is the cleanest, most balanced way to achieve pure intelligence boost while respecting the current stack.

Why Node C as Primary Spawn Point

- Node C already hosts Omi voice backend, Hermes enrichment, and Falcon vision. It is the natural perception + orchestration hub.

- Spawning from Node C allows real-time voice or screen input to trigger intelligence-boosting swarms without crossing to Node B for every task.
- Node B (Director) is kept pure for final high-fidelity reasoning and narrative synthesis — the swarm results are fed to it only when needed.

Why This Focuses on Pure Intelligence Boost

- The swarm amplifies **reasoning depth**, **knowledge synthesis**, **pattern extraction**, and **self-improvement** across the entire system.
- It turns isolated voice notes or RKG entries into rich, relational, multi-perspective intelligence that feeds back into MemPalace and the Akashik Artery.

HIGH-LEVEL DESIGN – SOVEREIGN INTELLIGENCE SWARM (Rust/Tokio on Node C)

Swarm Engine (`sovereign-intel-swarm` **Rust Crate)**
Spawned as a Tokio task on Node C (or delegated to Hermes Pod when available).

Core Components:

- **Archetype Agents**: 8–20 lightweight agents with distinct personas (Analyst, Synthesizer, Critic, Historian, Forecaster, etc.). Each runs as a Tokio task.
- **Graph Memory Layer**: Uses GBrain's MemPalace + SQLite-vec for relational knowledge. Agents communicate via typed edges.
- **Tokio Orchestration**: Async tasks with mpsc channels for inter-agent messaging and cancellation.
- **Dream Cycle Integration**: The swarm runs as a background "intelligence boost" job that extracts patterns and promotes high-signal insights into permanent memory.
- **Intelligence Boost Loop**:
    1. Input (voice note, screen capture, RKG query) arrives at Node C.
    2. Hermes validates and spawns swarm.
    3. Agents simulate multi-perspective reasoning in parallel.
    4. Results are synthesized and fed back to MemPalace / Akashik Artery.
    5. Node B (Director) receives the enriched context for final output if needed.

Nix Integration on Node C

```nix
# In oracle role (Node C)
services.sovereign-intel-swarm = {
  enable = true;
  package = pkgs.sovereign-intel-swarm;
  after = [ "hermes-supervisor.service" ];
  wantedBy = [ "multi-user.target" ];
};
```

Performance & Safety on Current Hardware

- Node C (RTX 2060 + 500 GB SSD) can handle 8–20 lightweight agents comfortably with Q4 models and Mooncake spillover.
- The 500 GB SSD stores swarm logs and temporary graph state.
- All swarm tasks are provenance-tagged and audited by Ouroboros.

Overall Effect on Sovereign Machina OS

The swarm turns the tri/quad-node system into a true **self-amplifying intelligence OS**. It boosts pure capability (deeper reasoning, richer synthesis, faster self-improvement) while staying fully local, sovereign, and respectful of the existing roles. Node C becomes the perception + swarm orchestration hub, Node B the final synthesizer, and Node A the shared memory reservoir.

ARCHITECT SIGN-OFF

Spawning the intelligence swarm from Node C (Oracle) is the logical and balanced choice for pure intelligence boost. It leverages the existing Omi + Hermes layer on Node C for input and enrichment, uses Tokio for async orchestration, and feeds results into GBrain / MemPalace for relational memory. This deepens the self-improving nature of the Sovereign Machina OS without touching Node A or Node B core logic. The swarm focuses on capability amplification (reasoning depth, pattern extraction, knowledge synthesis) while remaining lightweight and fully sovereign. When the Hermes Pod is available, it can optionally offload heavier swarm tasks for even better balance.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the starter Rust/Tokio swarm crate structure (`sovereign-intel-swarm`) with archetype agents and graph memory?
2. Shall I include the exact Nix service definition for spawning the swarm on Node C?

Reply with your priority and I will deliver the concrete starter code. The intelligence swarm is ready to be prototyped.